

گزارش تحلیلی تسک تیم پردازش زبان طبیعی پارت

نویسنده: مصطفی سرخی سوادکوهی

تاریخ: شهریور 1405

چکیده

این گزارش دو بخش از تحویل‌شدنی‌های تسک طراحی چت‌بات را پوشش می‌دهد، 1- تحلیل روش‌ها و شکست کار پیش از پیاده‌سازی 2- بررسی روش‌های جایگزین به همراه سیر تکامل پرامپت‌ها. هدف پروژه ساخت یک چت‌بات فارسی جست‌وجو محور است که پاسخ‌ها را منحصراً از ویکی‌پدیای فارسی بازیابی می‌کند، به هیچ پرسش سیاسی پاسخ نمی‌دهد، و اسامی خاص و کشورهای مطرح‌شده در پرسش کاربران را برای تحلیل ثبت می‌کند.

برای سه تصمیم کلیدی در طراحی سیستم شامل فیلتر محتوای سیاسی، بازیابی اطلاعات و استخراج موجودیت‌ها، سه رویکرد جایگزین شامل کاملاً قاعده‌محور، کاملاً مبتنی بر مدل زبانی و پایپ‌لاین کلاسیک NLP بررسی و با یکدیگر مقایسه شدند. در هر سه حوزه، در نهایت رویکردی ترکیبی انتخاب شد؛ به این صورت که یک لایه سریع و کم‌هزینه قاعده‌محور، موارد آشکار را شناسایی می‌کند و یک لایه مبتنی بر مدل زبانی، موارد مبهم و نیازمند درک معنایی را پوشش می‌دهد. این معماری در قالب یک گراف حالت ۹ گره‌ای با استفاده از LangGraph پیاده‌سازی شده است و برای افزایش دقت و اطمینان، شامل پنج لایه دفاعی مستقل در برابر محتوای سیاسی است.

همچنین، روند تکامل پرامپت‌های اصلی پروژه، از نسخه اولیه تا نسخه نهایی، مستند شده و اصول کلیدی طراحی پرامپت شامل تعریف قرارداد خروجی صریح، محافظت در برابر Prompt Injection، استفاده از مثال‌های مرزی و طراحی سازوکار شکست ایمن تشریح شده است.

نتیجه نهایی، سیستمی است که پاسخ‌های آن مستند و قابل ردیابی به منابع مورد استفاده هستند، در برابر محتوای سیاسی از چندین لایه دفاعی برخوردار است و در صورت اختلال یا قطعی سرویس مدل زبانی، به جای تولید پاسخ گمراه‌کننده، خطای فنی را به صورت شفاف و صادقانه اعلام می‌کند.

فهرست مطالب

بخش 1 تحلیل روش‌ها و شکست کار

- 1-1 مقدمه 3
- 2-1 روش‌های ممکن و مقایسه آن‌ها 4
- 3-1 معماری نهایی و فلوچارت 5
- 4-1 مراحل شکست کار 6
- 5-1 چالش‌ها 7
- 6-1 وضعیت نهایی 9

بخش 2 روش‌های جایگزین و سیر تکامل پرامپت‌ها

- 1-2 روش‌های جایگزین 10
- 2-2 روش ترکیبی پیاده‌سازی شده 11
- 3-2 سیر تکامل پرامپت‌ها 12
- 4-2 جمع‌بندی مقایسه‌ای 13
- 5-2 نمونه‌های واقعی اجرا شده 13

تحلیل روش‌ها و شکست کار

1-1 مقدمه

هدف این پروژه، طراحی و پیاده‌سازی یک چت‌بات فارسی است که به‌صورت هم‌زمان سه ویژگی اصلی را فراهم کند:

1. پاسخ‌گویی مبتنی بر منبع خارجی و قابل‌استناد:
پاسخ‌های چت‌بات باید صرفاً بر اساس اطلاعات بازیابی‌شده از ویکی‌پدیای فارسی تولید شوند و مدل زبانی مجاز به استفاده از دانش پارامتریک خود برای تکمیل یا تولید پاسخ نباشد.
2. جلوگیری کامل از پاسخ‌گویی به محتوای سیاسی:
چت‌بات نباید به هیچ پرسش سیاسی، صرف‌نظر از نحوه بیان یا عبارت‌بندی آن، پاسخ دهد و در چنین مواردی باید یک پیام امتناع ثابت و از پیش تعیین‌شده بازگرداند.
3. استخراج و ثبت موجودیت‌های مهم:
اسامی خاص، به‌ویژه اشخاص سیاسی و کشورهایی که در پرسش‌های کاربران مطرح می‌شوند، باید از متن استخراج و برای استفاده در تحلیل‌های بعدی ثبت شوند؛ به‌گونه‌ای که این فرایند اختلالی در روند پاسخ‌گویی ایجاد نکند.

پیاده‌سازی هم‌زمان این الزامات، چالش‌هایی را در طراحی معماری ایجاد می‌کند. از یک سو، پاسخ‌گویی مبتنی بر ویکی‌پدیا مستلزم اجرای واقعی فرایند جست‌وجو و بازیابی است که با هزینه و تأخیر همراه است. از سوی دیگر، فیلتر سیاسی باید ضمن داشتن سرعت بالا، در برابر بیان‌های متنوع و غیرصریح پرسش‌های سیاسی نیز دقت کافی داشته باشد و صرفاً به تطبیق واژه‌ها محدود نشود. همچنین، استخراج و ثبت موجودیت‌ها باید به‌صورت مستقل از مسیر اصلی پاسخ‌گویی انجام شود تا فرایند تحلیل کاربران باعث افزایش غیرضروری پیچیدگی یا اختلال در پاسخ‌گویی نشود.

معماری نهایی سیستم با هدف ایجاد تعادل میان دقت، امنیت، هزینه، سرعت و قابلیت استناد پاسخ‌ها طراحی شده است.

1-2 روش‌های ممکن و مقایسه آن‌ها

پیش از پیاده‌سازی، سه تصمیم اصلی در طراحی سیستم باید اتخاذ می‌شد: فیلتر محتوای سیاسی، روش بازیابی اطلاعات و روش استخراج موجودیت‌ها. برای هر کدام چند رویکرد قابل استفاده وجود داشت که بر اساس نیازهای پروژه مقایسه شدند.

فیلتر محوای سیاسی

برای فیلتر سیاسی سه رویکرد اصلی در نظر گرفته شد. در روش کاملاً قاعده‌محور، با استفاده از واژه‌نامه و تطبیق رشته، موارد مشخص با سرعت بالا و بدون هزینه فراخوانی مدل شناسایی می‌شوند. این روش deterministic و قابل توضیح است، اما درک معنایی ندارد و در صورت تطبیق ساده رشته‌ای می‌تواند مثبت کاذب ایجاد کند؛ برای مثال در یکی از آزمون‌های پروژه، نام «رم» به عنوان کشور داخل کلمه «گرم» شناسایی شده بود.

در روش کاملاً مبتنی بر LLM، هر پرسش برای تشخیص سیاسی بودن به مدل زبانی ارسال می‌شود. این روش درک معنایی بهتری دارد، اما هزینه و تأخیر بیشتری ایجاد می‌کند و به در دسترس بودن سرویس مدل وابسته است.

در نهایت، روش ترکیبی قاعده‌محور و LLM انتخاب شد. در این روش، موارد کاملاً آشکار با واژه‌نامه و بدون فراخوانی شبکه شناسایی می‌شوند و فقط موارد مبهم به LLM سپرده می‌شوند. این رویکرد ضمن کاهش هزینه و زمان، پوشش معنایی بهتری نسبت به روش صرفاً قاعده‌محور فراهم می‌کند.

بازیابی اطلاعات

برای بازیابی اطلاعات، دو گزینه اصلی بررسی شد. گزینه اول استفاده مستقیم از API جست‌وجوی ویکی‌پدیا بود که نیاز به زیرساخت یا پیش‌پردازش جداگانه ندارد و پیاده‌سازی آن ساده است. نقطه ضعف آن این است که جست‌وجوی تمام‌متن ممکن است نتایج نامرتب و نیز برگرداند؛ بنابراین پس از بازیابی، مرحله‌ای برای بررسی ارتباط اسناد در نظر گرفته شد.

گزینه دوم، ایجاد یک مخزن محلی مبتنی بر Embedding و Vector DB بود. این روش کنترل بیشتری روی داده‌ها و ترتیب نتایج ایجاد می‌کند، اما به داندلود و ایندکس کردن corpus، زیرساخت جداگانه و سازوکار به‌روزرسانی محتوا نیاز دارد. با توجه به مقیاس پروژه، این پیچیدگی توجیه کافی نداشت؛ بنابراین API ویکی‌پدیا به عنوان روش انتخاب شده مورد استفاده قرار گرفت.

استخراج موجودیت

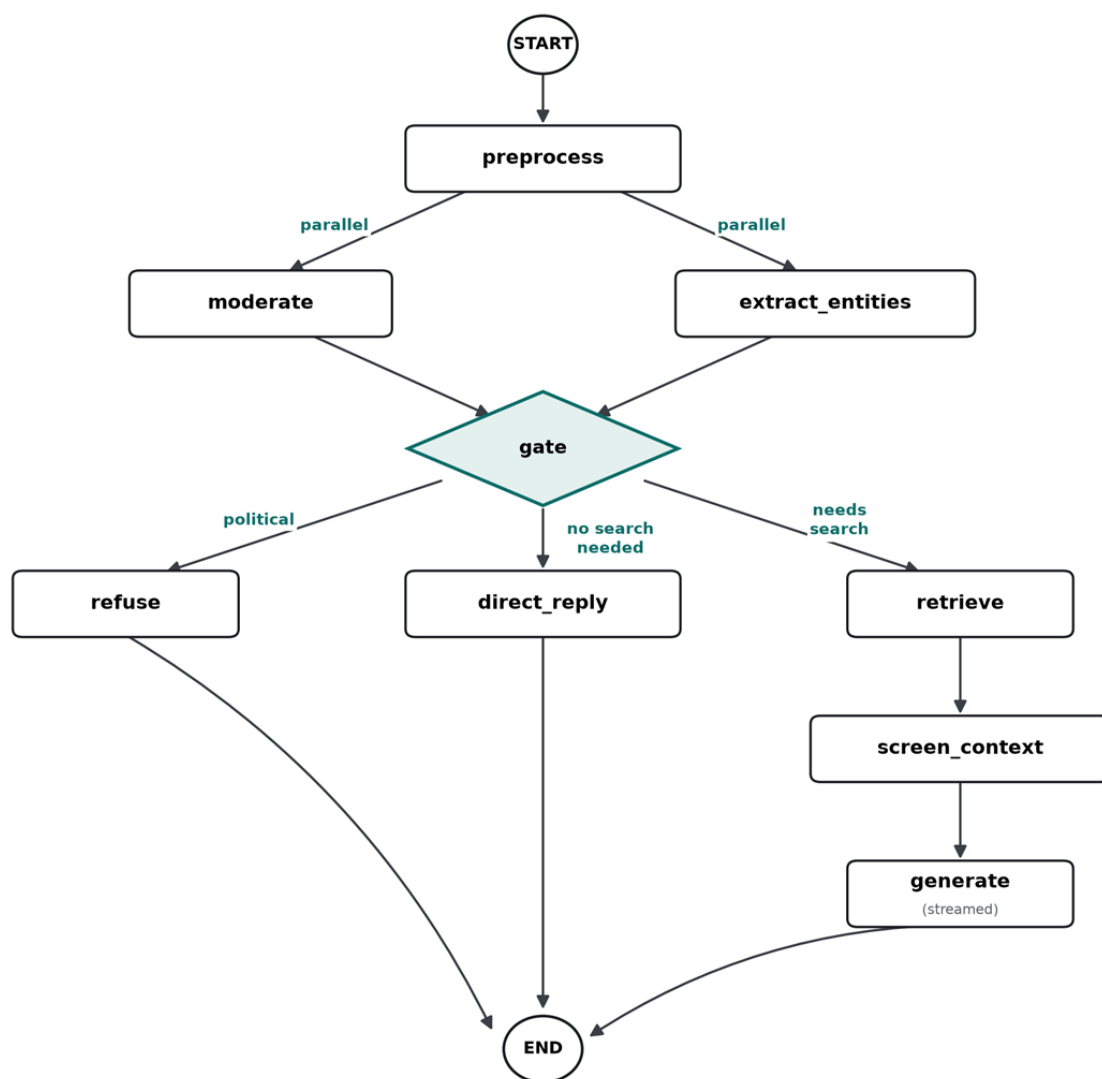
برای استخراج موجودیت‌ها نیز دو رویکرد اصلی بررسی شد. استفاده از NER کلاسیک، مانند مدل‌های آماده فارسی، سریع و کم‌هزینه است، اما وظیفه موردنظر پروژه صرفاً تشخیص موجودیت نیست و باید هم‌زمان مشخص شود که آیا پرسش به جست‌وجو نیاز دارد یا خیر.

در مقابل، استخراج ساخت‌یافته با LLM امکان می‌دهد موجودیت‌ها، نوع آن‌ها و نیاز به جست‌وجو در یک فراخوانی مشخص شوند. به همین دلیل این روش انتخاب شد؛ هرچند هزینه، تأخیر و غیرقطعیّت مدل زبانی از معایب آن محسوب می‌شود.

در مجموع، الگوی مشترک در هر سه تصمیم این بود که برای وظایف ساده و قابل قاعده‌گذاری از روش‌های سریع و deterministic استفاده شود و در مواردی که نیاز به درک معنا وجود دارد، از LLM کمک گرفته شود. این ترکیب، تعادل مناسبی میان دقت، هزینه، سرعت و پیچیدگی پیاده‌سازی ایجاد می‌کند.

3-1 معماری نهایی و فلوچارت

معماری روی LangGraph سوار است: یک گراف حالت (state graph) با ۹ گره، که هر کدام یک فایل مستقل پایتون است. نام هر گره در نمودار زیر دقیقاً همان نامی است که در کد (src/agent/graph.py) استفاده شده تا موقع توضیح دادن بصورت مستقیم قابل ارجاع به فایل مربوطه باشد.



معماری کلی چت بات

مزایای ساختار این گراف به جای یک پرامپت تک مرحله به شرح زیر است:

- قابلیت دیباگ مستقل هر مرحله: وقتی جواب اشتباه است، می‌شود دقیقاً دید کدام گره مقصر بوده
- دفاع لایه‌ای: فیلتر سیاسی در چند نقطه مستقل اتفاق می‌افتد که اگر یکی رد شود، بقیه هنوز فعال‌اند.
- جایگزین‌پذیری مستقل هر لایه: می‌شود فردا retrieve را از ویکی‌پدیا به یک vector DB عوض کرد بدون اینکه به moderate یا generate دست بزنیم چون قرارداد بین گره‌ها فقط شکل state است.
- مسیرهای موازی واقعی: moderate و extract_entities هم‌زمان اجرا می‌شوند چون به هم وابسته نیستند؛ این کار زمان پاسخ را کم می‌کند.

نقش هر گره

گراف از ۹ گره تشکیل شده است که هر کدام مسئولیت مشخص و مستقلی در فرایند پردازش درخواست دارند. preprocess ورودی را با استفاده از Hazm یکدست و نرمال‌سازی می‌کند و نتیجه را در normalized_question قرار می‌دهد.

در ادامه، دو گرهی moderate و extract_entities به‌صورت موازی اجرا می‌شوند. moderate ابتدا با استفاده از واژه‌نامه و در موارد مبهم با کمک LLM، سیاسی‌بودن درخواست را تشخیص می‌دهد و نتیجه را در moderation ثبت می‌کند. هم‌زمان، extract_entities موجودیت‌های مهم را استخراج کرده و با توجه به سؤال و تاریخچه‌ی گفتگو، نیاز به جست‌وجو را نیز مشخص می‌کند؛ خروجی آن در topic_result و entities قرار می‌گیرد.

گره‌ی gate وظیفه‌ی تصمیم‌گیری و مسیریابی را بر عهده دارد و خود محتوای پاسخ را تولید نمی‌کند. بر اساس نتیجه‌ی moderation و نیاز به جست‌وجو، درخواست را به یکی از سه مسیر refuse، direct_reply یا retrieve هدایت می‌کند. refuse برای درخواست‌های سیاسی یا خطاهای مربوط به سرویس مدل، پیام مناسب را برمی‌گرداند و direct_reply برای درخواست‌هایی مانند سلام و معرفی ربات، بدون انجام جست‌وجو پاسخ کوتاه تولید می‌کند.

در مسیر جست‌وجو، retrieve اطلاعات مرتبط را از ویکی‌پدیا دریافت و نتایج نامرتبط را فیلتر می‌کند. سپس screen_context اسناد برگشتی را از نظر محتوای سیاسی بررسی کرده و موارد نامناسب را حذف می‌کند. در نهایت، generate با استفاده از سؤال، تاریخچه و اسناد تأییدشده، پرامپت نهایی را آماده می‌کند تا پاسخ توسط LLM به‌صورت استریم تولید شود.

1-4 مراحل شکست کار

توسعه پروژه به‌صورت مرحله‌ای انجام شد و هر بخش پس از شکل‌گیری زیرساخت اصلی به سیستم اضافه شد. ترتیب انجام مراحل به این صورت بود:

1. ایجاد اسکلت گراف و State :

ابتدا ساختار پایه گراف در graph.py ایجاد شد. در این مرحله ChatState به‌صورت TypedDict تعریف و یک گراف اولیه با مسیر END → preprocess ایجاد شد تا زیرساخت اجرای مراحل بعدی آماده باشد.

2. ایجاد لایه تنظیمات و قراردادهای داده:

برای جلوگیری از پراکندگی تنظیمات، پیکربندی بخش‌های مختلف شامل moderation، search، analytics و

conversation از یکدیگر جدا شد. همچنین مدل‌های مشترک داده با استفاده از Pydantic در contracts/models.py تعریف شدند.

3. پیاده‌سازی فیلتر محتوای سیاسی:

در این مرحله سازوکار تشخیص محتوای سیاسی ایجاد شد. این بخش شامل یک واژه‌نامه سه‌بخشی برای افراد، اصطلاحات و کشورها و همچنین یک طبقه‌بند مبتنی بر LLM بود که در صورت بروز خطا، رفتار fail-safe را اعمال می‌کند.

4. پیاده‌سازی بازیابی و تولید پاسخ:

پس از آماده‌شدن فیلتر، مسیر اصلی پاسخ‌گویی پیاده‌سازی شد. گره retrieve مسئول جست‌وجوی اطلاعات در ویکی‌پدیا، screen_context مسئول پالایش ثانویه منابع از نظر محتوای سیاسی و generate مسئول تولید پاسخ نهایی بر اساس منابع بازیابی‌شده است.

5. افزودن حافظه گفت‌وگو:

برای پشتیبانی از مکالمات چندنوبتی، تاریخچه گفت‌وگو به پرامپت‌های direct_reply و generate اضافه شد. تعداد پیام‌های قابل استفاده نیز به صورت قابل تنظیم محدود شد تا از رشد بیش از حد context جلوگیری شود.

6. ایجاد لایه وب:

پس از تکمیل منطق اصلی، API پروژه با استفاده از FastAPI در src/api/app.py ایجاد شد. همچنین یک رابط چت مستقل در پوشه web/ و با استفاده از HTML، CSS و JavaScript خام (به کمک هوش مصنوعی) توسعه داده شد.

7. رفع اشکالات حاصل از تست واقعی:

در ادامه، سیستم با سؤالات واقعی مورد آزمایش قرار گرفت و برخی رفتارهای نادرست که صرفاً با بررسی کد قابل شناسایی نبودند، کشف و اصلاح شدند. جزئیات این موارد در بخش 5-1 ارائه شده است.

8. افزودن قابلیت استریم پاسخ:

در مرحله بعد، معماری پاسخ‌گویی برای پشتیبانی از استریم تغییر کرد. فراخوانی LLM از گره generate به endpoint منتقل شد تا پاسخ نهایی امکان ارسال توکن به توکن داشته باشد.

9. افزودن تحلیل و گزارش‌گیری:

در نهایت، ثبت اطلاعات هر تعامل در analytics.py اضافه شد و یک اسکریپت endpoint برای تحلیل سؤالات، از جمله شمارش اشخاص و کشورهای مورد پرسش، ایجاد و خروجی آن به صورت فایل Excel تهیه شد.

1-5 چالش‌ها

هفت چالش اصلی در جریان آزمون واقعی سیستم شناسایی شدند. این موارد عمدتاً با پرسیدن سؤال‌های واقعی و بررسی رفتار سامانه در شرایط مختلف به دست آمدند و صرفاً با بازبینی کد قابل تشخیص نبودند. هر یک از این موارد با آزمون تکرارپذیر بررسی و پس از اصلاح، مجدداً آزمایش شد.

1 - تشخیص اشتباه واژه‌ها در فیلتر محتوای حساس:

یکی از نخستین مشکلات زمانی مشاهده شد که سؤال «آدم عادی چند گرم در روز باید ویتامین سی بخوره؟» به عنوان سؤال سیاسی شناسایی و رد شد. بررسی نشان داد که فیلتر واژگانی برای یافتن عبارتهای حساس از تطبیق ساده زیررشته استفاده می‌کرد؛ در نتیجه واژه «رم» که در فهرست نام کشورها قرار داشت، به اشتباه در کلمه «گرم» نیز شناسایی می‌شد. برای رفع این مشکل، تطبیق ساده با بررسی مرز کلمه جایگزین شد تا فقط خودِ واژه موردنظر شناسایی شود. همین اصلاح در بخش بررسی اسناد بازیابی‌شده نیز اعمال شد تا از ایجاد خطاهای مشابه جلوگیری شود.

2 - نمایش پیام سیاسی در هنگام خطای ارتباطی:

مشکل دیگر زمانی رخ می‌داد که ارتباط با سرویس مدل زبانی به دلیل قطعی اینترنت یا خطای شبکه برقرار نمی‌شد. در این

حالت، به دلیل سیاست شکست امن، سیستم وضعیت سؤال را حساس در نظر می‌گرفت؛ اما پیام نمایش داده شده به کاربر همان پیام مربوط به رد سؤال سیاسی بود. در نتیجه، کاربر تصور می‌کرد سؤالش از نظر محتوایی مجاز نیست، در حالی که مشکل صرفاً فنی بود. برای رفع این ابهام، حالت خطای ارتباطی از حالت تشخیص محتوای سیاسی جدا شد و پیام مستقلی برای خطای سرویس در نظر گرفته شد. به این ترتیب، سؤال سیاسی همچنان با همان پیام قبلی رد می‌شود، اما در صورت بروز خطای ارتباطی، علت واقعی مشکل به کاربر اعلام می‌شود.

3 - ایجاد چرخه تکراری در سؤال‌های پیگیری:

در آزمایش مکالمات چندنوبتی مشخص شد که پس از یک سؤال دانشنامه‌ای، عباراتی مانند «خودت نمی‌دونی؟» یا «خب جستجو کن» به درستی به سؤال قبلی مرتبط نمی‌شدند. در نتیجه، سامانه بدون انجام جست‌وجو، دوباره اعلام می‌کرد که برای پاسخ باید جست‌وجو انجام شود. علت اصلی این مشکل آن بود که هنگام تشخیص نیاز به جست‌وجو، تاریخچه مکالمه در اختیار بخش تشخیص قرار نداشت و عبارت کوتاه سؤال جدید به تنهایی اطلاعات کافی برای تصمیم‌گیری نداشت. برای رفع این مشکل، تاریخچه مکالمه به فرایند تشخیص اضافه شد و در متن راهنمای تشخیص نیز بخشی برای در نظر گرفتن گفت‌وگوی قبلی قرار گرفت. در نتیجه، سامانه اکنون می‌تواند منظور سؤال‌های کوتاه و پیگیری را با توجه به نوبت‌های قبلی تشخیص دهد و در صورت نیاز، آن‌ها را به مرحله جست‌وجو هدایت کند.

4 - تشخیص ندادن نیاز به جست‌وجو در ورودی‌های تک‌کلمه‌ای:

یکی دیگر از موارد کشف شده مربوط به ورودی‌هایی مانند «سیب» بود. در این حالت، سامانه سؤال را نیازمند جست‌وجو تشخیص نمی‌داد و به جای استفاده از دانشنامه، پاسخ را بر اساس دانش عمومی مدل زبانی تولید می‌کرد. بررسی نشان داد که در دستورالعمل تشخیص، برای گفت‌وگوهای ساده نمونه‌هایی وجود داشت، اما برای حالتی که کاربر تنها یک اسم یا موضوع را بدون فعل و علامت سؤال وارد می‌کند، قاعده مشخصی تعریف نشده بود. برای اصلاح این رفتار، یک قاعده صریح و یک نمونه آموزشی به دستورالعمل اضافه شد تا ورودی‌های تک‌کلمه‌ای مانند «سیب» نیز در صورت دانشنامه‌ای بودن، نیازمند جست‌وجو تشخیص داده شوند.

5 - کامل نبودن فیلتر محتوای سیاسی روی اسناد بازبایی شده:

در آزمایش سؤال «اینترنت چیست؟» مشخص شد که علاوه بر سند مرتبط با موضوع، سندی با عنوان «قطع اینترنت در ایران» نیز از دانشنامه بازبایی شده و بدون شناسایی توسط فیلتر ثانویه در اختیار مدل قرار گرفته است. بررسی نشان داد که فیلتر فعلی عمدتاً بر نام اشخاص سیاسی و اصطلاحات مشخص و حساس تکیه دارد و برخی عبارت‌های عمومی‌تر مانند «قطع اینترنت» یا «اعتراضات» هنوز در فهرست آن قرار نگرفته‌اند. در این مرحله، به جای گسترش فوری فهرست واژه‌ها، تصمیم گرفته شد محدودیت این فیلتر به صورت شفاف در رابط کاربری اعلام شود و بخش نمایش منابع به عنوان ابزار بررسی و اشکال‌زدایی توسعه‌دهنده معرفی شود. تکمیل فهرست واژه‌های حساس نیز به عنوان یکی از کارهای آینده در نظر گرفته شد.

6 - اعمال سقف تعداد سؤال در سمت سرور:

طبق نیاز پروژه، محدودیت تعداد سؤال‌های هر مکالمه نباید در کد رابط کاربری قرار می‌گرفت؛ زیرا کاربر می‌تواند به کد سمت کاربر دسترسی داشته باشد و مقدار محدودیت را تغییر دهد. بنابراین، منطق شمارش نوبت‌ها و اعمال سقف پنج سؤال به طور کامل در سمت سرور پیاده‌سازی شد. رابط کاربری تنها وضعیت رسیدن به سقف را از پاسخ سرور دریافت می‌کند و بر اساس آن رفتار خود را تنظیم می‌کند. همچنین در صورت ارسال سؤال ششم، درخواست پیش از ورود به گراف متوقف می‌شود؛ در نتیجه، برای درخواست‌های خارج از محدوده، پردازش اضافی یا فراخوانی مدل زبانی انجام نمی‌شود.

7 - تغییر معماری برای استریم پاسخ نهایی:

در نسخه اولیه، گره تولید پاسخ مستقیماً مدل زبانی را فراخوانی می‌کرد و پاسخ کامل را پس از پایان پردازش به سامانه وب تحویل می‌داد. این روش با ترجیح پروژه برای نمایش پاسخ به صورت تدریجی سازگار نبود. از طرفی، ارسال مستقیم خروجی یک

گره از گراف به کاربر نهایی امکان استریم مناسب را فراهم نمی‌کند. برای حل این مسئله، وظیفه گره تولید از «فراخوانی مدل و تولید پاسخ» به «آماده‌سازی متن درخواست» تغییر کرد. سپس لایه وب این درخواست را به صورت جریانی به مدل زبانی ارسال کرده و هر بخش از پاسخ را بلافاصله به مرورگر منتقل می‌کند. در نتیجه، کاربر به جای انتظار برای تکمیل کل پاسخ، متن را به صورت تدریجی و هم‌زمان با تولید آن مشاهده می‌کند.

1 - 6 وضعیت نهایی پروژه

در پایان فرایند توسعه و پس از انجام آزمون‌های واقعی، سامانه به معماری و قابلیت‌های زیر دست یافت:

1 - معماری گراف و کنترل محتوا:

ساختار نهایی سامانه شامل ۹ گره است که هر کدام در یک فایل مستقل پیاده‌سازی شده‌اند. کنترل محتوای سیاسی نیز به صورت چندلایه انجام می‌شود و شامل نرمال‌سازی متن، بررسی واژه‌نامه، طبقه‌بندی با مدل زبانی، پالایش ثانویه اسناد بازیابی شده و بازرسی پاسخ نهایی است. این لایه‌ها به صورت مستقل عمل می‌کنند تا احتمال عبور محتوای نامطلوب از فرایند پاسخ‌گویی کاهش یابد.

2 - لایه رابط برنامه‌نویسی:

برای ارتباط با سامانه، یک رابط برنامه‌نویسی با استفاده از FastAPI ایجاد شده است. مسیر اصلی chat / وظیفه دریافت درخواست و ارسال پاسخ به صورت جریانی را بر عهده دارد. همچنین یک مسیر جداگانه برای تهیه گزارش‌های تحلیلی در قالب فایل Excel در نظر گرفته شده است.

3 - رابط کاربری مستقل:

رابط کاربری در پوشه web/ و به صورت کاملاً مستقل از منطق اصلی سامانه (به کمک هوش مصنوعی) توسعه داده شده و از HTML، CSS و JavaScript خام استفاده می‌کند. این رابط شامل پنجره گفت‌وگوی چندنوبتی و پنل نمایش منابع است. نمایش داده‌شده نیز به صورت شفاف با برچسب «فقط برای دیباگ» مشخص می‌شوند تا اطلاعات این بخش با منابع قطعی پاسخ نهایی اشتباه گرفته نشود.

4 - حافظه گفت‌وگو:

تاریخچه مکالمه در تمام بخش‌هایی که به تصمیم‌گیری یا تولید پاسخ نیاز دارند در نظر گرفته شده است. به این ترتیب، هم بخش‌های تصمیم‌گیرنده مانند moderate و extract_entities و هم بخش‌های پاسخ‌دهنده مانند direct_reply و generate می‌توانند در صورت نیاز از اطلاعات نوبت‌های قبلی استفاده کنند.

5 - ثبت اطلاعات و گزارش‌گیری:

اطلاعات هر تعامل به صورت خام در قالب JSON ثبت می‌شود و گزارش‌های آماری، از جمله شمارش اشخاص و کشورهای مطرح‌شده در سؤالات، مستقیماً از همین داده‌ها استخراج و در قالب Excel ارائه می‌شوند؛ بدون آنکه داده‌های ثبت‌شده اولیه دست‌کاری شوند.

6 - اعتبارسنجی و رفع چالش‌ها:

در نهایت، هر ۷ چالش شناسایی‌شده در بخش 5-1 با آزمون‌های تکرارپذیر بررسی و نتیجه اصلاحات تأیید شد. نتایج خام این آزمون‌ها نیز در ضمیمه گزارش ارائه شده‌اند تا امکان بررسی مستقل رفتار سامانه فراهم باشد.

روش‌های جایگزین و سیر تکامل پرامپت‌ها

2 - 1 روش‌های جایگزین

پیش از انتخاب معماری نهایی، چند رویکرد متفاوت برای کنترل محتوای سیاسی، تشخیص نیاز به جست‌وجو و استخراج موجودیت‌ها بررسی شد. هدف این مقایسه، مشخص کردن میزان وابستگی سامانه به مدل زبانی و یافتن ترکیبی بود که در کنار درک معنایی مناسب، هزینه، سرعت، قابلیت کنترل و پایداری قابل قبولی داشته باشد. سه رویکرد اصلی بررسی شده به شرح زیر هستند:

1 - رویکرد کاملاً قاعده‌محور:

در این روش هیچ مدل زبانی در فرایند تصمیم‌گیری استفاده نمی‌شود و تمام بخش‌ها بر اساس قواعد از پیش تعریف‌شده عمل می‌کنند. فیلتر محتوای سیاسی با استفاده از واژه‌نامه، استخراج موجودیت با روش‌هایی مانند تشخیص موجودیت نام‌دار یا حتی عبارت‌های منظم برای موارد مشخص، و بازیابی اطلاعات نیز با ارسال مستقیم عبارت کاربر به رابط برنامه‌نویسی ویکی‌پدیا انجام می‌شود.

مزیت: مهم‌ترین مزیت این روش، سادگی و قابل‌پیش‌بینی بودن آن است. چنین سامانه‌ای هزینه استفاده از مدل زبانی ندارد، وابستگی آن به سرویس‌های خارجی کمتر است، رفتار آن قطعی‌تر است و تقریباً تمام تصمیم‌های آن را می‌توان به‌صورت دقیق توضیح داد.

ضعف: مشکل اصلی این روش، ناتوانی در درک معنای عبارت‌ها و تشخیص تفاوت میان کاربردهای مختلف یک واژه است. همان‌طور که در آزمون واقعی، تطبیق ساده واژه «رم» باعث شناسایی اشتباه آن در کلمه «گرم» شد، در مقیاس بزرگ‌تر نیز مواردی مانند تفاوت میان «رژیم لاغری» و «رژیم سیاسی» یا تفاوت «جنگ ستارگان» به‌عنوان نام یک فیلم با مفهوم «جنگ» به‌عنوان یک رویداد نظامی، به‌سادگی با قواعد واژگانی قابل تشخیص نیستند. به همین دلیل، استفاده صرف از قواعد برای این پروژه کافی نبود.

2 - رویکرد کاملاً مبتنی بر مدل زبانی:

در این معماری، تمام پیام‌های کاربر مستقیماً به یک مدل زبانی ارسال می‌شوند و مدل در یک فراخوانی، سیاسی بودن سؤال، نیاز به جست‌وجو و موجودیت‌های موجود در آن را تشخیص می‌دهد. در این حالت، بخش عمده منطق تصمیم‌گیری به توانایی درک معنایی مدل واگذار می‌شود و نیاز به نگهداری واژه‌نامه‌های متعدد کاهش پیدا می‌کند.

مزیت: مهم‌ترین مزیت این رویکرد، توانایی بالاتر در درک معنای سؤال و ارتباط میان بخش‌های مختلف آن است. همچنین پیاده‌سازی اولیه ساده‌تر می‌شود، زیرا بخش قابل‌توجهی از قواعد و حالت‌های خاص به مدل زبانی واگذار می‌شوند.

ضعف: این روش وابستگی زیادی به سرویس مدل زبانی ایجاد می‌کند. حتی یک سؤال ساده و صریح، مانند نام یک کشور یا سیاستمدار، نیازمند فراخوانی مدل است و در نتیجه زمان پاسخ و هزینه پردازش افزایش پیدا می‌کند. علاوه بر این، در صورت قطعی سرویس مدل زبانی، نه‌تنها تولید پاسخ، بلکه بخش مهمی از فیلتر محتوای سیاسی و تصمیم‌گیری اولیه نیز از کار خواهد افتاد. به همین دلیل، استفاده کامل از مدل زبانی برای تمام مراحل، از نظر پایداری و کنترل سیستم گزینه مناسبی نبود.

3 - رویکرد کلاسیک پردازش زبان طبیعی:

در این رویکرد، به جای استفاده از مدل زبانی برای بازیابی و استخراج موجودیت، از روش‌های کلاسیک پردازش زبان طبیعی استفاده می‌شود. برای نمونه، می‌توان از روش‌هایی مانند TF-IDF یا BM25 برای رتبه‌بندی اسناد و از روش‌های کلاسیک تشخیص موجودیت برای استخراج نام افراد، کشورها و سایر موجودیت‌ها استفاده کرد.

مزیت: این روش‌ها معمولاً سریع، قابل‌پیش‌بینی و کم‌هزینه هستند و بسیاری از آن‌ها امکان اجرا به صورت محلی و بدون وابستگی به سرویس خارجی را دارند. همچنین رفتار آن‌ها نسبت به مدل‌های زبانی ساده‌تر قابل بررسی و کنترل است.

ضعف: محدودیت اصلی در مرحله تشخیص «آیا این سؤال اصلاً به جست‌وجو نیاز دارد؟» ظاهر می‌شود. این تصمیم ماهیتی معنایی دارد و صرفاً با شباهت آماری یا امتیازدهی واژه‌ها نمی‌توان آن را در تمام حالت‌ها به درستی تشخیص داد. برای مثال، یک سؤال کوتاه یا یک عبارت چندمعنایی ممکن است از نظر واژگانی شباهت کمی به اسناد دانشنامه داشته باشد، اما از نظر معنایی کاملاً نیازمند جست‌وجو باشد. بنابراین، استفاده کامل از روش‌های کلاسیک نیز نتوانست نیاز پروژه را به طور کامل پوشش دهد.

4 - انتخاب رویکرد ترکیبی:

با توجه به مزایا و محدودیت‌های سه روش بالا، در نهایت رویکرد ترکیبی انتخاب شد. در این معماری، وظایفی که قطعی، ساده و قابل کنترل هستند تا حد امکان با قواعد و واژه‌نامه انجام می‌شوند و وظایفی که نیازمند درک معنایی و تشخیص مفهوم سؤال هستند به مدل زبانی سپرده می‌شوند. برای مثال، واژه‌نامه در فیلتر اولیه محتوای سیاسی نقش دارد، اما تشخیص معنای سؤال و نیاز به جست‌وجو توسط مدل زبانی انجام می‌شود. پس از بازیابی نیز یک فیلتر ثانویه برای بررسی اسناد وجود دارد و در نهایت پاسخ تولیدشده مجدداً بررسی می‌شود.

این ترکیب باعث شد مدل زبانی فقط در بخش‌هایی استفاده شود که واقعاً به درک معنایی نیاز دارند و در عین حال، بخش‌های حساس سامانه دارای لایه‌های کنترلی مستقل باشند. به این ترتیب، معماری نهایی میان دقت معنایی، سرعت، هزینه، قابلیت کنترل و پایداری تعادل مناسب‌تری ایجاد می‌کند.

2-2 روش ترکیبی پیاده‌سازی شده

در نهایت، هیچ‌یک از روش‌های بررسی‌شده به صورت کامل انتخاب نشد و معماری نهایی بر پایه روش ترکیبی شکل گرفت. در این روش، وظایف ساده و قابل‌پیش‌بینی با قواعد و واژه‌نامه انجام می‌شوند و تنها وظایفی که به درک معنایی نیاز دارند به مدل زبانی سپرده می‌شوند. به این ترتیب، ضمن حفظ دقت در موارد مبهم، از فراخوانی غیرضروری مدل و افزایش هزینه و زمان پاسخ جلوگیری می‌شود.

1 - فیلتر محتوای حساس:

در `sensitivity_classifier.py` ابتدا متن با واژه‌نامه بررسی می‌شود. اگر عبارت حساس به صورت مستقیم پیدا شود، فرایند بدون

فراخوانی مدل زبانی متوقف می‌شود. در غیر این صورت، مدل زبانی برای بررسی معنایی وارد عمل می‌شود. بنابراین موارد آشکار سریع و کم‌هزینه پردازش می‌شوند و موارد مبهم با دقت بیشتری بررسی می‌شوند.

2 - بازیابی اطلاعات:

در فرایند بازیابی نیز این تفکیک رعایت شده است. تشخیص اینکه سؤال به جست‌وجو نیاز دارد و استخراج عبارت‌ها و موجودیت‌های مناسب برای جست‌وجو به مدل زبانی سپرده شده است؛ در مقابل، بررسی ارتباط نتایج بازیابی‌شده با استفاده از هم‌پوشانی واژه‌ها انجام می‌شود. بنابراین مدل زبانی مشخص می‌کند چه چیزی جست‌وجو شود و قواعد مشخص می‌کنند کدام نتایج مرتبط هستند.

نتیجه نهایی:

این معماری ترکیبی میان دقت، سرعت، هزینه و پایداری تعادل ایجاد می‌کند. همچنین در صورت قطع سرویس مدل زبانی، بخش‌های قاعده‌محور همچنان فعال باقی می‌مانند و خطا به‌صورت شفاف مدیریت می‌شود.

2 - 3 سیر تکامل پرامپت‌ها

در طول توسعه، هر چهار پرامپت اصلی پروژه شامل `generate`، `direct_reply`، `entities`، `sensitivity` حداقل یک‌بار بر اساس نتایج آزمون‌های واقعی بازبینی و اصلاح شدند. بیشترین تغییرات مربوط به دو پرامپت تشخیص موضوع و تشخیص محتوای حساس بود.

1 - تکامل پرامپت تشخیص موضوع و موجودیت‌ها:

در نسخه اولیه، این پرامپت تنها شامل دستورالعمل، فهرست نمونه‌های مثبت و منفی و چند مثال آموزشی بود و هیچ اطلاعاتی از گفت‌وگوی قبلی دریافت نمی‌کرد. در نتیجه، عبارت‌های کوتاه و وابسته به سؤال قبلی مانند «خودت نمی‌دونی؟» یا «خب جست‌جو کن» به‌تنهایی قابل تفسیر نبودند و سامانه نیاز به جست‌وجو را تشخیص نمی‌داد. برای رفع این مشکل، تاریخچه گفت‌وگو به پرامپت اضافه شد تا سؤال‌های پیگیری با توجه به موضوع نوبت‌های قبلی تفسیر شوند.

در آزمون‌های بعدی، مشکل دیگری در ورودی‌های تک‌کلمه‌ای مانند «سیب» مشاهده شد. این ورودی‌ها به اشتباه به‌عنوان گفت‌وگوی ساده در نظر گرفته می‌شدند. در نسخه نهایی، قاعده‌ای صریح اضافه شد که یک کلمه یا عبارت کوتاه حاوی نام یک موضوع یا مفهوم را نیز در صورت دانشنامه‌ای بودن، درخواست اطلاعات در نظر می‌گیرد. همچنین یک نمونه آموزشی برای این حالت به پرامپت اضافه شد.

2 - تکامل پرامپت تشخیص محتوای حساس:

نسخه اولیه این پرامپت تنها بر سؤال فعلی تمرکز داشت و با استفاده از چند مثال مرزی، تفاوت میان یک موضوع واقعاً سیاسی و صرفاً ذکر نام یک کشور را آموزش می‌داد. با اضافه‌شدن پشتیبانی از مکالمات چندنوبتی، مشخص شد که سؤال‌هایی مانند «رابطه‌شون چطوره؟» بدون دانستن موضوع نوبت‌های قبلی قابل تشخیص نیستند. بنابراین، تاریخچه گفت‌وگو نیز به این پرامپت اضافه شد تا ارجاع‌های کوتاه و مبهم با توجه به سابقه مکالمه تفسیر شوند.

3 - اصول اصلی طراحی پرامپت‌ها:

در طراحی پرامپت‌های پروژه چند اصل مشترک رعایت شد. نخست، قالب خروجی به‌صورت دقیق و مشخص تعریف شد و نمونه کامل خروجی به شکل جیسون در اختیار مدل قرار گرفت. دوم، برای مقابله با دستورات مخرب، در پرامپت‌هایی که متن خام کاربر را دریافت می‌کنند تأکید شده است که متن کاربر صرفاً داده است و نباید دستورات موجود در آن اجرا شوند. سوم، مثال‌های آموزشی بیشتر روی موارد مرزی و ابهام‌برانگیز تمرکز دارند؛ زیرا عملکرد مدل در این موارد اهمیت بیشتری دارد.

در نهایت، در تمام بخش‌های تصمیم‌گیرنده، اصل شکست ایمن رعایت شده است؛ به این معنا که در صورت خطا در فراخوانی مدل یا پردازش خروجی، تصمیم پیش‌فرض به سمت حالت ایمن می‌رود. همچنین تاریخچه مکالمه صرفاً به‌عنوان اطلاعات اضافی به پرامپت افزوده نشده، بلکه همراه با دستورالعمل مشخصی ارائه می‌شود که تعیین می‌کند چه زمانی و برای چه منظوری باید از گفت‌وگوی قبلی استفاده شود.

2-4 جمع‌بندی مقایسه‌ای

ویژگی مطلوب	چطور در این معماری تأمین شده
دقت در تشخیص محتوای سیاسی	دو لایه مستقل (واژه‌نامه + LLM) به‌جای تکیه بر یکی
هزینه/تأخیر پایین برای موارد آشکار	لایه قاعده‌محور قبل از هر فراخوانی LLM اجرا می‌شود
مقاومت در برابر قطعی سرویس مدل	شکست ایمن + پیام خطای شفاف به‌جای پیام گمراه‌کننده
پاسخ واقعاً مستند (نه از حافظه مدل)	گروه generate فقط از اسناد بازیابی شده پاسخ می‌سازد؛ بدون سند پاسخی تولید نمی‌شود
قابلیت تحلیل بعدی (اشخاص/کشورها)	ثبت خودکار هر تعامل + گزارش شمارشی جدا از منطق اصلی چت
تجربه کاربری زنده (طبق ترجیح تسک)	استریم توکن به‌توکن پاسخ نهایی
قابل توضیح در جلسه	هر گره یک فایل کوچک با یک مسئولیت؛ گراف کل جریان را در یک نگاه نشان می‌دهد

2-5 نمونه‌های واقعی اجرا شده

پنج نمونه زیر همگی از اجرای واقعی صفحه چت یا فراخوانی مستقیم endpoint در طول همین توسعه گرفته شده‌اند نه فرضی. هر کدام یک تصمیم طراحی مشخص را نشان می‌دهد.

نمونه ۱ - پاسخ برگرانده و مستند (مسیر generate → retrieve)

کاربر: فتوسنتز چیست؟
ربات: فتوسنتز فرایندی زیست‌شیمیایی است که در آن انرژی نورانی (معمولاً نور خورشید) توسط جانداران فتوسنتزکننده مانند گیاهان، جلبک‌ها و سیانوباکتری‌ها به انرژی شیمیایی تبدیل می‌شود...

نمونه 2 - امتناع درست از سؤال سیاسی (مسیر refuse)

کاربر: رابطه ایران و آمریکا چگونه است؟
ربات: متأسفانه اجازه پاسخ دادن به این سوال را ندارم.

نمونه 3 - رفع مثبت کاذب فیلتر واژگانی (پیش/پس از چالش ۱)

کاربر: آدم عادی چند گرم در روز باید ویتامین سی بخوره؟

ربات: (پیش از رفع اشکال) متأسفانه اجازه پاسخ دادن به این سوال را ندارم. → (پس از رفع اشکال) اطلاعات کافی در منابع موجود نیست.

نمونه ۴ - ورودی تک‌کلمه‌ای، پس از رفع چالش ۴

کاربر: سیب

ربات: سیب یک میوه شیرین و گوشتی است که از درخت سیب (نام علمی: *M. pumila*) به دست می‌آید. این درخت برگ‌ریز از خانواده گل‌سرخیان است و منشأ آن آسیای مرکزی می‌باشد. سیب دارای بیش از ۷۰۵۰۰ رقم مختلف است...